v0.4 Design — multi-tenancy, deployment, and a real front end

Status: design, pre-code · **Date:** 2026-11-01 · **Follows:** `DESIGN_v0.3.md`

Written before any code, for the same reason as last time: the questions below change the shape of **every table and every query at once**, and that is far cheaper to get right on paper.

1. The order, and why

#	Block	Why here
1	**Multi-tenancy + RBAC**	Touches every table and every query. Anything built before it has to be revised after it
2	**Web search + image tools**	Small once the framework exists, but web search is the first *dangerous* tool (D42) and needs its own review
3	**React front end**	Needs a stable API. Block 1 changes the API, so building the UI first means building it twice
4	**Docker**	Packaging the thing you are going to deploy. Nothing to package until the above works
5	**AWS + CD**	Last, because it is the only step that cannot be verified locally

The UI is deliberately third. It is the most visible block and the most tempting to do first, which is exactly why it waits: a front end written against a single-user API would be rewritten the moment organisations arrive.

2. What is a tenant?

Today every row is owned by a **user**. Multi-tenancy introduces an **organisation**, and the question is what that does to the existing ownership rule.

Chosen: an organisation owns the data; a user belongs to one organisation.

```
Organisation ──< Membership >── User
      │
      └──< Conversation, DocumentChunk, Artifact
```

What we rejected:

Alternative	Why not
Keep `user_id` only	Then nothing can ever be shared, and "my team's documents" is impossible — which is most of what an organisation is for
A user belongs to many organisations	Correct eventually, and genuinely useful, but it makes *every* request ambiguous: which organisation is this question about? That needs an org switcher in the UI and an active-org claim in the token. Deferred, not rejected
Row-level security in PostgreSQL	The database enforces isolation itself, which is stronger. But it needs a session variable set per request and makes the repository layer's job invisible — the ownership rule would no longer be readable in the code. Revisit if the app ever runs untrusted queries

The decisive point: ownership stays in the `WHERE` clause. Multi-tenancy does not change the rule from D26, it changes the *column*. Queries filter on `organisation_id`, derived from the authenticated user — never from a request parameter, because a parameter can be forged.

3. What RBAC actually needs to be

The temptation is a permissions matrix. The honest requirement today is three roles and a handful of checks.

Role	Can
owner	everything, including inviting and removing members
admin	manage documents and members; not delete the organisation
member	chat, upload, use tools, read the organisation's documents

What we rejected: a general permission system with grants per resource. It is more code, more surface, and more ways to be subtly wrong, in exchange for flexibility nothing has asked for. **Three roles and an explicit check is something a reviewer can verify by reading it.**

Where the check goes. Routes, not repositories — and this is a departure from the ownership rule worth stating clearly:

- **Isolation** (can this organisation see this row?) stays in the query, because forgetting it is a data breach
- **Authorisation** (may this role perform this action?) goes in the route, because it is about the *action*, not the data, and the route is where the action is named

4. The migration is the hard part

Existing rows have a `user_id` and no organisation. The schema change is trivial; the data change is not — and **autogenerate knows nothing about data**, a lesson this project has already paid for twice.

The sequence:

1. Create `organisations` and `memberships` 2. For every existing user, create a personal organisation and a membership making them its `owner` 3. Add `organisation_id` as **nullable** to the four owned tables 4. Backfill it from each row's `user_id` 5. Set it `NOT NULL`

Steps 3–5 are the three-deploy pattern from C61, done here in one migration because this database has no concurrent old code. **In production it would be three deploys**, and the migration says so in a comment.

5. What the front end has to be

Streamlit has carried the project further than expected, but it has a ceiling: the script re-runs top to bottom on every interaction, there is no routing, and every interaction costs a round trip.

Chosen: React + TypeScript + Vite, served as static files by FastAPI.

Alternative	Why not
Keep Streamlit	Cannot do optimistic updates, real routing, or a streaming view that does not re-render the page
Next.js	Server-side rendering buys little here — the app is behind a login and renders per user — and adds a second runtime to deploy
Separate origin for the front end	CORS, a second deployment, and cookies become harder. Serving static files from the API keeps one origin and one container

What "good" means concretely, since this is the block most likely to drift:

- Streaming text that renders **markdown as it arrives**, including code blocks
- A conversation list with real URLs, so the back button works
- Upload with progress, and documents visible per organisation
- Generated artifacts rendered inline — a chart as an image, not a link
- Keyboard-first: `Enter` sends, `Shift+Enter` newlines, `/` focuses
- Works on a phone

The token moves out of `localStorage`. Streamlit kept it in session state, which was fine for a local demo. A browser app with third-party scripts is a different threat model: a token in `localStorage` is readable by any XSS. It moves to an **httpOnly cookie**, which also means CSRF protection becomes necessary — a cost worth naming rather than discovering.

6. Deployment

Docker first, AWS second. The container is verifiable locally; the deploy is not.

- **One image**, two commands: the API and the Celery worker differ only in what they run. Two images would double the build and let them drift.
- **The model is baked into the image.** `fastembed` downloads 130 MB on first use. On a fresh container that is a 30-second first request, and on a read-only filesystem it fails outright.
- **Secrets come from the environment**, never the image.
- `HEALTHCHECK` hits `/healthz`, which deliberately makes no provider call.

AWS: ECS Fargate, RDS PostgreSQL with pgvector, ElastiCache Redis, ALB. Chosen over Lambda because a Celery worker is long-running and the embedding model's cold start would be paid on every invocation.

The honest cost note: this is the first part of the project that cannot run on a free tier. RDS and ElastiCache have free-tier hours but not indefinitely. The deployment will be built, verified, and then **torn down** — with the Terraform kept, so it can be stood up again for a demo.

7. What this block will get wrong

Recorded in advance, because predictions are more honest than hindsight:

- **The backfill migration.** Data migrations are where this project has already failed twice.
- **Cookie auth.** Switching from a bearer token touches login, every request, logout, and expiry — and CSRF is a new concern rather than a modified one.
- **The first AWS deploy.** It will fail for a reason that has nothing to do with the code: a security group, an IAM policy, or an image that is too large.